

Авторские права
© Postgres Professional, 2017–2024
Авторы: Егор Рогов, Павел Лузанов, Илья Баштанов, Игорь Гнатюк
Фото: Олег Бартунов (монастырь Пху и пик Бхрикути, Непал)

Использование материалов курса
Некоммерческое использование материалов курса (презентации, демонстрации) разрешается без ограничений. Коммерческое использование возможно только с письменного разрешения компании Postgres Professional. Запрещается внесение изменений в материалы курса.

Обратная связь
Отзывы, замечания и предложения направляйте по адресу:
edu@postgrespro.ru

Отказ от ответственности
Компания Postgres Professional не несет никакой ответственности за любые повреждения и убытки, включая потерю дохода, нанесенные прямым или непрямым, специальным или случайным использованием материалов курса. Компания Postgres Professional не предоставляет каких-либо гарантий на материалы курса. Материалы курса предоставляются на основе принципа «как есть» и компания Postgres Professional не обязана предоставлять сопровождение, поддержку, обновления, расширения и изменения.

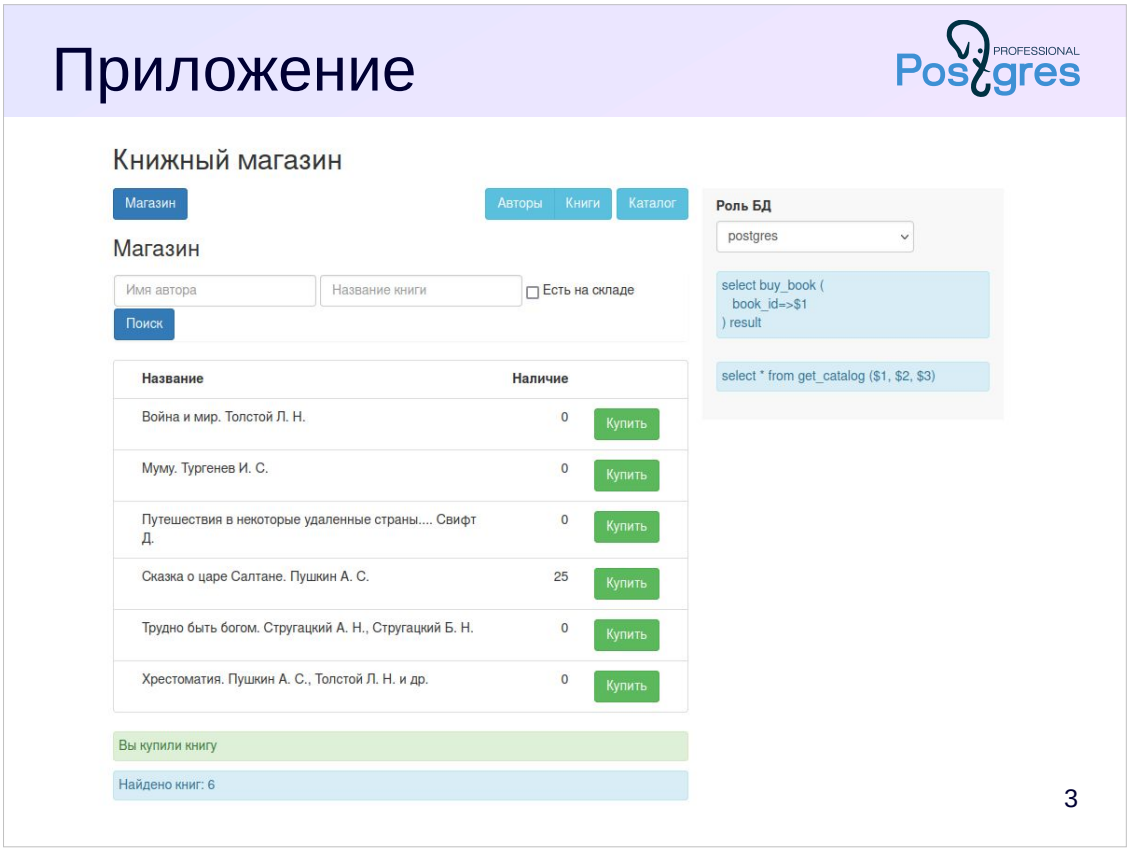
Темы

Обзор приложения «Книжный магазин»

Проектирование схемы данных, нормализация

Итоговая схема данных приложения

Организация интерфейса между клиентом и сервером



Приложение состоит из нескольких частей, представленных вкладками.

«Магазин» — это интерфейс веб-пользователя, в котором он может покупать книги.

Остальные вкладки соответствуют внутреннему интерфейсу, доступному только сотрудникам («админка» сайта).

«Каталог» — интерфейс кладовщика, в котором он может заказывать закупку книг на склад магазина и просматривать операции поступлений и покупок.

«Книги» и «Авторы» — интерфейс библиотекаря, в котором он может регистрировать авторов и их книги.

В учебных целях вся функциональность представлена на одной общей веб-странице. Если какая-то часть функциональности недоступна из-за того, что на сервере нет подходящего объекта (таблицы, функции и т. п.), приложение сообщит об этом. Также приложение выводит текст запросов, которые оно посылает на сервер.

В ходе курса мы начнем с пустой базы данных и постепенно реализуем в ней все необходимые компоненты.

P. S. Исходный код приложения не является темой курса, но может быть получен в репозитории https://pubgit.postgrespro.ru/pub/dev1app.git

Демонстрация приложения

В этой демонстрации мы показываем приложение «Книжный магазин» в том виде, в котором оно будет после завершения всех практических заданий. Приложение откроется в отдельной вкладке браузера виртуальной машины курса:

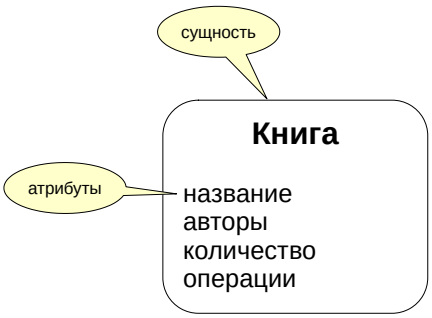
Открываем http://localhost...

```
student$ xdg-open http://localhost
```

Книги

ER-модель для высокоуровневого проектирования

сущности — понятия предметной области
связи — отношения между сущностями
атрибуты — свойства сущностей и связей

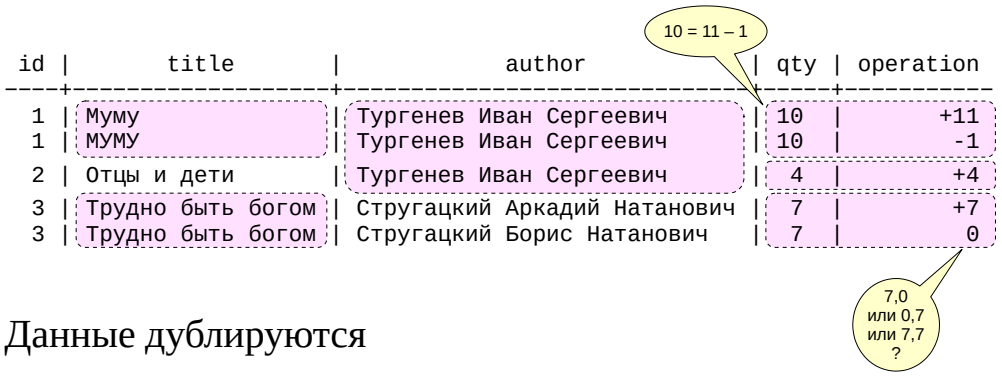


После того, как мы посмотрели на внешний вид приложения и поняли его функциональность, нам нужно разобраться со схемой данных.
Мы не будем хоть сколько-нибудь глубоко вникать в вопросы проектирования баз данных — это отдельная дисциплина, не входящая в данный курс. Но совсем обойти вниманием эту тему тоже нельзя.

Часто для высокоуровневого проектирования баз данных используется модель «сущность — связи», или ER-модель (Entity — Relationship). Она оперирует *сущностями* (понятиями предметной области), *связями* между ними и *атрибутами* (свойствами сущностей и связей). Модель позволяет рассуждать на логическом уровне, не опускаясь до деталей представления данных на физическом (в виде таблиц).

Первым подходом к проектированию может служить диаграмма, представленная на слайде: можно представить одну большую сущность «Книга», а все остальное сделать ее атрибутами.

Схема данных



Данные дублируются

- сложно поддерживать согласованность
- сложно обновлять
- сложно писать запросы

Очевидно, что это неправильный подход. На диаграмме это может быть не так заметно, но давайте попробуем отобразить диаграмму на таблицы БД. Это можно сделать разными способами, например так, как показано на слайде: сущность становится таблицей, ее атрибуты — столбцами этой таблицы.

Здесь хорошо видно, что часть данных дублируется (эти фрагменты выделены на рисунке). Дублирование приводит к сложностям с обеспечением согласованности — а это едва ли не главная задача СУБД.

Например, каждая из двух строк, относящихся к книге 3, должна содержать общее количество (7 штук). Что нужно сделать, чтобы отразить покупку книги? С одной стороны, надо добавить строки для отражения операции покупки (а сколько строк добавлять, еще две?). С другой, во всех строках надо уменьшить количество с 7 до 6. А что делать, если в результате какой-нибудь ошибки значение количества в одной из этих строк будет отличаться от значения в другой строке? Как сформулировать ограничение целостности, которое запрещает такую ситуацию?

Также усложняются и многие запросы. Как найти общее число книг? Как найти всех уникальных авторов?

Итак, такая схема плохо работает для реляционных баз данных.

Схема данных (вариант)

```
 entity | attribute  | value
--------+------------+------------------------
      1 | title      | Муму
      1 | author     | Тургенев Иван Сергеевич
      1 | qty        | 10
      1 | operation  | +11
      1 | operation  | -1
      2 | title      | Отцы и дети
      2 | author     | Тургенев Иван Сергеевич
      2 | qty        | 4
      2 | operation  | +4
    ... | ...        | ...
```

Данные без схемы

- поддержка согласованности на стороне приложения
- сложно писать запросы
- низкая производительность (множественные соединения)

Другой вариант отображения сущности в таблицу — так называемая схема EAV: сущность — атрибут — значение. Она позволяет уложить вообще все, что угодно, в одну-единственную таблицу. Формально мы получаем реляционную базу данных, а фактически здесь отсутствует схема и СУБД не может гарантировать согласованность данных. Поддержка согласованности данных целиком ложится на приложение, что, безусловно, рано или поздно приведет к ее нарушению.

В такой схеме сложно писать запросы (хотя и довольно просто их генерировать), и в результате работа со сколько-нибудь большим объемом данных становится проблемой из-за постоянных многократных соединений таблицы самой с собой.

Однако следует отметить, что такой подход имеет и плюсы. Например, очевидно, что такую «схему» данных легко наращивать: при добавлении новой сущности или атрибута мы просто добавляем в нашу таблицу новые строки.

И все-таки так делать не стоит.

Схема данных (вариант)

```
 book_id |                        description
---------+-----------------------------------------------------------
       1 | {"title": "Муму",
         |  "authors": [ "Тургенев Иван Сергеевич" ],
         |  "qty": 10,
         |  "operations": [ +11, -1 ]}
       3 | {"title": "Трудно быть богом",
         |  "authors": [ "Стругацкий Аркадий Натанович",
         |               "Стругацкий Борис Натанович" ],
         |  "qty": 7,
         |  "operations": [ +7 ]}
     ... | ...
```

Данные без схемы

- поддержка согласованности на стороне приложения
- сложно писать запросы (специальный язык)
- индексная поддержка есть

Еще одна вариация на ту же тему — представление данных в виде JSON, в духе NoSQL. По сути, тут применимы все замечания, сделанные ранее.

Кроме того, запросы к такой структуре придется писать не на SQL, а используя какой-то специализированный язык (раньше скорее всего выбор пал бы на jsQuery, а в PostgreSQL 16 удобно пользоваться возможностями SQL/JSONPath, определенными в стандарте SQL:2016).

https://github.com/postgrespro/jsquery

https://postgrespro.ru/docs/postgresql/16/functions-json#FUNCTIONS-SQLJSON-PATH

Хотя у PostgreSQL есть индексная поддержка JSON, производительность все равно под вопросом.

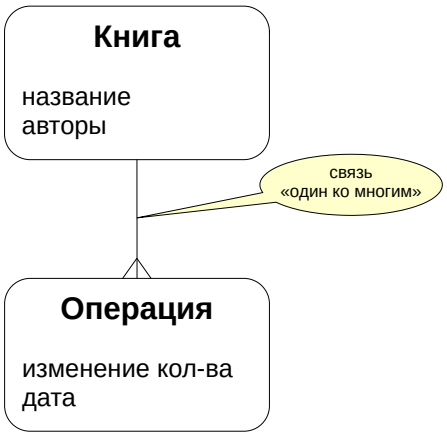
Такую схему удобно применять, когда от базы данных требуется только получение JSON по идентификатору, но не требуется серьезная работа с данными *внутри* JSON. Это не наш случай.

(Разумеется, это не категоричное утверждение. См. последнее задание в практике.)

Книги и операции

Нормализация — уменьшение избыточности данных

разбиение крупных сущностей на более мелкие



Итак, нам требуется устранить избыточность данных, чтобы привести их в вид, удобный для обработки в реляционной СУБД. Этот процесс называется нормализацией.

Возможно, вы знакомы с понятиями *нормальных форм* (первая, вторая, третья, Бойса-Кодда и т. д.). Мы не будем говорить о них; на неформальном уровне достаточно понимать, что весь этот математический аппарат преследует одну-единственную цель: устранение избыточности.

Средство для уменьшения избыточности — разбиение большой сущности на несколько меньших. Как именно это сделать, подскажет здравый смысл (который все равно нельзя заменить одним только знанием нормальных форм).

В нашем случае все достаточно просто. Давайте начнем с отделения книг от операций. Эти две сущности связаны отношением «один ко многим»: каждая книга может иметь несколько операций, но каждая операция относится только к одной книге.

Схема данных

books

book_id	title	author
1	Муму	Тургенев Иван Сергеевич
2	Отцы и дети	Тургенев Иван Сергеевич
3	Трудно быть богом	Стругацкий Аркадий Натанович
3	Трудно быть богом	Стругацкий Борис Натанович

operations

operation_id	book_id	qty_change	date_created
1	1	+10	2020-07-13
2	1	-1	2020-08-25
3	3	+7	2020-07-13
4	2	+4	2020-07-13

На уровне структуры базы данных это может быть представлено двумя таблицами: книги (books) и операции (operations).

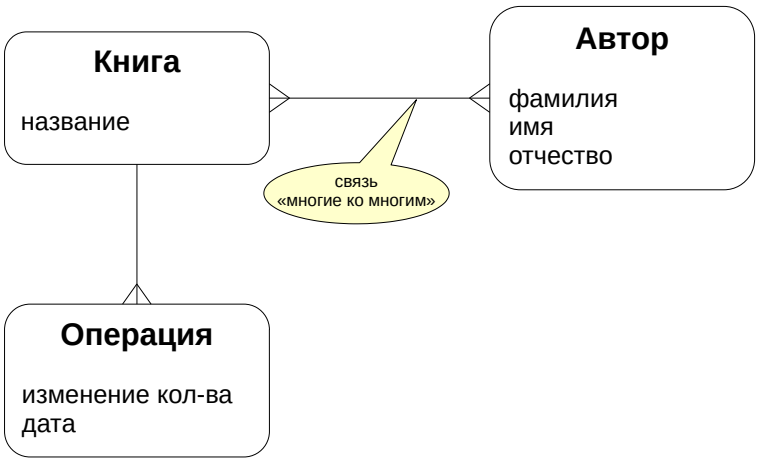
Операция состоит в изменении количества книг (положительном при заказе; отрицательном при покупке). Заметьте, что теперь у книги нет атрибута «количество». Вместо этого достаточно просуммировать изменения количества по всем операциям, относящимся к данной книге. Дополнительный атрибут «количество» у книги снова привел бы к избыточности данных.

Не исключено, что такое решение вас насторожило. Удобно ли вместо простого обращения к полю подсчитывать сумму? Но мы можем создать представление, которое для каждой книги будет показывать количество. Это не приведет к появлению избыточности, поскольку представление — это всего лишь запрос.

Второй момент — производительность. Если подсчет суммы приведет к ухудшению производительности, мы можем выполнить обратный процесс — денормализацию: добавить в таблицу книг поле «количество» и обеспечить его согласованность с таблицей операций. Надо ли этим заниматься — вопрос, ответ на который выходит за рамки этого курса (он рассматривается в курсе QPT «Оптимизация запросов»). Но из общих соображений понятно, что для нашего «игрушечного» магазина это не требуется. Однако мы еще вернемся к вопросу денормализации в теме «Триггеры».

Итак, как видно на слайде, выделение операций в отдельную сущность решило часть проблем дублирования, но не все.

Книги, авторы и операции



Поэтому надо сделать еще один шаг: отделить авторов от книг и связать их соотношением «многие ко многим»: и каждая книга может быть написана несколькими авторами, и каждый автор может написать несколько книг. На уровне таблиц такая связь реализуется с помощью дополнительной промежуточной таблицы.

Атрибутами автора можно сделать фамилию, имя и отчество. Это имеет смысл, поскольку нам может потребоваться работать отдельно с каждым из этих атрибутов, например, выводить фамилию и инициалы.

Схема данных приложения

```
student$ psql bookstore
```

Итак, схема данных приложения состоит из четырех таблиц:

```
=> \dt
```

```
            List of relations
  Schema   |    Name    | Type  |  Owner
-----------+------------+-------+---------
 bookstore | authors    | table | student
 bookstore | authorship | table | student
 bookstore | books      | table | student
 bookstore | operations | table | student
(4 rows)
```

Книги

```
=> \d books
```

```
                        Table "bookstore.books"
 Column  |  Type   | Collation | Nullable |           Default
---------+---------+-----------+----------+-----------------------------
 book_id | integer |           | not null | generated always as identity
 title   | text    |           | not null |
Indexes:
    "books_pkey" PRIMARY KEY, btree (book_id)
Referenced by:
    TABLE "authorship" CONSTRAINT "authorship_book_id_fkey" FOREIGN KEY (book_id)
REFERENCES books(book_id)
    TABLE "operations" CONSTRAINT "operations_book_id_fkey" FOREIGN KEY (book_id)
REFERENCES books(book_id)
```

Здесь мы используем типы данных:

- integer — целое число;
- text — символьный, текстовая строка произвольной длины.

А также используем ограничение целостности:

- PRIMARY KEY — первичный ключ.

Конструкция GENERATED AS IDENTITY служит для автоматической генерации уникальных значений (в версиях до 10 для этого использовался псевдотип serial).

Строки столбцов, объявленных как GENERATED AS IDENTITY, получают значения из специальных объектов базы данных — последовательностей. Имя использованной последовательности можно узнать так:

```
=> SELECT pg_get_serial_sequence('books','book_id');
```

```
   pg_get_serial_sequence
----------------------------
 bookstore.books_book_id_seq
(1 row)
```

Последовательности можно при необходимости создавать вручную, а также обращаться к ним непосредственно:

```
=> SELECT nextval('books_book_id_seq');
```

```
 nextval
---------
       7
(1 row)
```

Последовательность — самый эффективный способ генерации уникальных номеров. Но следует иметь в виду, что последовательность не гарантирует:

- отсутствия пропусков в нумерации (так как изменение происходит не транзакционно);
- монотонного возрастания номеров (если используется кеширование значений в сеансах).

Вот какие данные находятся в таблице книг:

```
=> SELECT * FROM books \gx
```

```
-[ RECORD 1
]-----------------------------------------------------------------------------------------------
-----------------------------------------------------
book_id | 1
title   | Сказка о царе Салтане
-[ RECORD 2
]-----------------------------------------------------------------------------------------------
-----------------------------------------------------
book_id | 2
title   | Муму
-[ RECORD 3
]-----------------------------------------------------------------------------------------------
-----------------------------------------------------
book_id | 3
title   | Трудно быть богом
-[ RECORD 4
]-----------------------------------------------------------------------------------------------
-----------------------------------------------------
book_id | 4
title   | Война и мир
-[ RECORD 5
]-----------------------------------------------------------------------------------------------
-----------------------------------------------------
book_id | 5
title   | Путешествия в некоторые удаленные страны мира в четырех частях: сочинение
Лемюэля Гулливера, сначала хирурга, а затем капитана нескольких кораблей
-[ RECORD 6
]-----------------------------------------------------------------------------------------------
-----------------------------------------------------
book_id | 6
title   | Хрестоматия
```

Обратите внимание, что названия могут быть длинными. Мы используем расширенный формат вывода psql, а появившийся в PostgreSQL 16 параметр xheader_width поможет обрезать строку заголовка "широкого" столбца:

```
=> \pset xheader_width 40
```

```
Expanded header width is 40.
```

Сравните вывод:

```
=> SELECT * FROM books \gx
```

```
-[ RECORD 1 ]--------------------------
book_id | 1
title   | Сказка о царе Салтане
-[ RECORD 2 ]--------------------------
book_id | 2
title   | Муму
-[ RECORD 3 ]--------------------------
book_id | 3
title   | Трудно быть богом
-[ RECORD 4 ]--------------------------
book_id | 4
title   | Война и мир
-[ RECORD 5 ]--------------------------
book_id | 5
title   | Путешествия в некоторые удаленные страны мира в четырех частях: сочинение
Лемюэля Гулливера, сначала хирурга, а затем капитана нескольких кораблей
-[ RECORD 6 ]--------------------------
book_id | 6
title   | Хрестоматия
```

Авторы

```
=> \d authors
```

```
                        Table "bookstore.authors"
   Column    |  Type   | Collation | Nullable |           Default
-------------+---------+-----------+----------+-----------------------------
 author_id   | integer |           | not null | generated always as identity
 last_name   | text    |           | not null |
 first_name  | text    |           | not null |
 middle_name | text    |           |          |
Indexes:
    "authors_pkey" PRIMARY KEY, btree (author_id)
Referenced by:
    TABLE "authorship" CONSTRAINT "authorship_author_id_fkey" FOREIGN KEY (author_id)
REFERENCES authors(author_id)
```

Здесь дополнительно используется ограничение целостности:

- NOT NULL — обязательность, то есть недопустимость неопределенных значений.

```
=> SELECT * FROM authors;
```

```
 author_id | last_name  | first_name | middle_name
-----------+------------+------------+-------------
         1 | Пушкин     | Александр  | Сергеевич
         2 | Тургенев   | Иван       | Сергеевич
         3 | Стругацкий | Борис      | Натанович
         4 | Стругацкий | Аркадий    | Натанович
         5 | Толстой    | Лев        | Николаевич
         6 | Свифт      | Джонатан   |
(6 rows)
```

Обратите внимание, что отчество может отсутствовать (или быть заданным пустой строкой).

Ограничение PRIMARY KEY в выводе команды \d упоминалось вместе со словами «индекс» и «btree».

Btree (B-дерево) — основной тип индекса, используемый в базах данных для ускорения поиска и для поддержки ограничений целостности (первичного ключа и уникальности).

Представим себе, что в магазине продаются книги миллиона авторов-однофамильцев:

```
=> BEGIN; -- начнем транзакцию, чтобы откатить потом изменения
```

```
BEGIN
```

```
=> INSERT INTO authors(first_name, last_name)
    SELECT 'Графоман', 'Графоманов' FROM generate_series(1, 1_000_000);
```

```
INSERT 0 1000000
```

Сколько времени занимает поиск одного автора в такой таблице?

```
=> \timing on
```

```
Timing is on.
```

```
=> SELECT * FROM authors WHERE last_name = 'Пушкин';
```

```
 author_id | last_name | first_name | middle_name
-----------+-----------+------------+-------------
         1 | Пушкин    | Александр  | Сергеевич
(1 row)

Time: 72,261 ms
```

```
=> \timing off
```

```
Timing is off.
```

Если попросить оптимизатор показать план запроса, мы увидим в нем Seq Scan — последовательное сканирование всей таблицы в поисках нужного значения (Filter):

```
=> EXPLAIN (costs off)
SELECT * FROM authors WHERE last_name = 'Пушкин';
```

```
               QUERY PLAN
----------------------------------------
 Seq Scan on authors
   Filter: (last_name = 'Пушкин'::text)
(2 rows)
```

А если искать по полю, которое проиндексировано?

```
=> \timing on
```

```
Timing is on.
```

```
=> SELECT * FROM authors WHERE author_id = 1;
```

```
 author_id | last_name | first_name | middle_name
-----------+-----------+------------+-------------
         1 | Пушкин    | Александр  | Сергеевич
(1 row)

Time: 0,200 ms
```

```
=> \timing off
```

```
Timing is off.
```

Время уменьшилось на несколько порядков.

А в плане запроса появился индекс:

```
=> EXPLAIN (costs off)
SELECT * FROM authors WHERE author_id = 1;
```

```
                QUERY PLAN
----------------------------------------
 Index Scan using authors_pkey on authors
   Index Cond: (author_id = 1)
(2 rows)
```

Можно создать индекс и по фамилии (и проанализировать таблицу, чтобы собрать актуальную статистику):

```
=> ANALYZE authors;
```

```
ANALYZE
```

```
=> CREATE INDEX ON authors(last_name);
```

```
CREATE INDEX
```

```
=> EXPLAIN (costs off)
SELECT * FROM authors WHERE last_name = 'Пушкин';
```

```
                    QUERY PLAN
--------------------------------------------------
 Index Scan using authors_last_name_idx on authors
   Index Cond: (last_name = 'Пушкин'::text)
(2 rows)
```

Однако индекс не является универсальным средством увеличения производительности. Обычно индекс очень полезен, если из таблицы требуется выбрать небольшую долю всех имеющихся строк. Если нужно прочитать много данных, индекс будет только мешать, и оптимизатор это понимает:

```
=> EXPLAIN (costs off)
SELECT * FROM authors WHERE last_name = 'Графоманов';
```

```
                 QUERY PLAN
-----------------------------------------
 Seq Scan on authors
   Filter: (last_name = 'Графоманов'::text)
(2 rows)
```

Кроме того, надо учитывать накладные расходы на обновление индексов при изменении таблицы и занимаемое ими место на диске.

Отменим все наши изменения (включая создание индекса):

```
=> ROLLBACK;
```

```
ROLLBACK
```

```
=> ANALYZE authors;
```

```
ANALYZE
```

Авторства

С помощью этой таблицы реализуется связь «многие ко многим».

```
=> \d authorship
```

```
            Table "bookstore.authorship"
  Column   |  Type   | Collation | Nullable | Default
-----------+---------+-----------+----------+---------
 book_id   | integer |           | not null |
 author_id | integer |           | not null |
 seq_num   | integer |           | not null |
Indexes:
    "authorship_pkey" PRIMARY KEY, btree (book_id, author_id)
Foreign-key constraints:
    "authorship_author_id_fkey" FOREIGN KEY (author_id) REFERENCES authors(author_id)
    "authorship_book_id_fkey" FOREIGN KEY (book_id) REFERENCES books(book_id)
```

Здесь к уже использованным ограничениям добавляется ограничение ссылочной целостности:

- FOREIGN KEY — внешний ключ.

Фактически таблица содержит два внешних ключа: один ссылается на таблицу книг, другой — на таблицу авторов.

Столбец seq_num определяет последовательность, в которой должны перечисляться авторы одной книги, если их несколько.

Также обратите внимание на то, что первичный ключ — составной.

```
=> SELECT * FROM authorship;
```

```
 book_id | author_id | seq_num
---------+-----------+---------
       1 |         1 |       1
       2 |         2 |       1
       3 |         3 |       2
       3 |         4 |       1
       4 |         5 |       1
       5 |         6 |       1
       6 |         1 |       1
       6 |         5 |       2
       6 |         2 |       3
(9 rows)
```

Операции

```
=> \d operations
```

```
                        Table "bookstore.operations"
    Column    |  Type   | Collation | Nullable |           Default
--------------+---------+-----------+----------+-----------------------------
 operation_id | integer |           | not null | generated always as identity
 book_id      | integer |           | not null |
 qty_change   | integer |           | not null |
 date_created | date    |           | not null | CURRENT_DATE
Indexes:
    "operations_pkey" PRIMARY KEY, btree (operation_id)
Foreign-key constraints:
    "operations_book_id_fkey" FOREIGN KEY (book_id) REFERENCES books(book_id)
```

В этой таблице используется еще один тип данных:

- date — дата (без указания времени).

Для столбца date_created указано значение по умолчанию (DEFAULT) — текущая дата.

```
=> SELECT * FROM operations;
```

```
 operation_id | book_id | qty_change | date_created
--------------+---------+------------+--------------
            1 |       1 |         10 | 2025-06-24
            2 |       1 |         10 | 2025-06-24
            3 |       1 |         -1 | 2025-06-24
(3 rows)
```

Кроме тех типов данных, которые используются в таблицах приложения, мы постоянно будем встречаться с логическим типом (boolean). Например, такой тип имеют логические выражения в условии WHERE.

Важно помнить, что, в отличие от традиционных языков программирования, SQL использует трехзначную логику: кроме значений true и false, логическое значение может принимать неопределенное значение NULL (которое можно понимать как «значение неизвестно»).

В примерах встречаются и некоторые другие типы данных, информацию о работе с ними можно найти в материалах курса («Основные типы данных и функции», datatypes.pdf).

Кроме того, в курсе мы подробно рассмотрим более сложные типы:

- составной — записи, аналогичные строкам таблиц (в теме «SQL. Составные типы»);
- массивы (в теме «PL/pgSQL. Массивы»).

Организация интерфейса

Таблицы и триггеры

чтение данных напрямую из таблицы (представления);
запись данных напрямую в таблицу (представление),
плюс триггеры для изменения связанных таблиц

приложение должно быть в курсе модели данных,
максимальная гибкость

сложно поддерживать согласованность

Функции

чтение данных из табличных функций;
запись данных через вызов функций

приложение отделено от модели данных и ограничено API

большой объем работы по изготовлению функций-оберток,
потенциальные проблемы с производительностью

Есть несколько способов организации интерфейса между клиентской и серверной частями приложения.

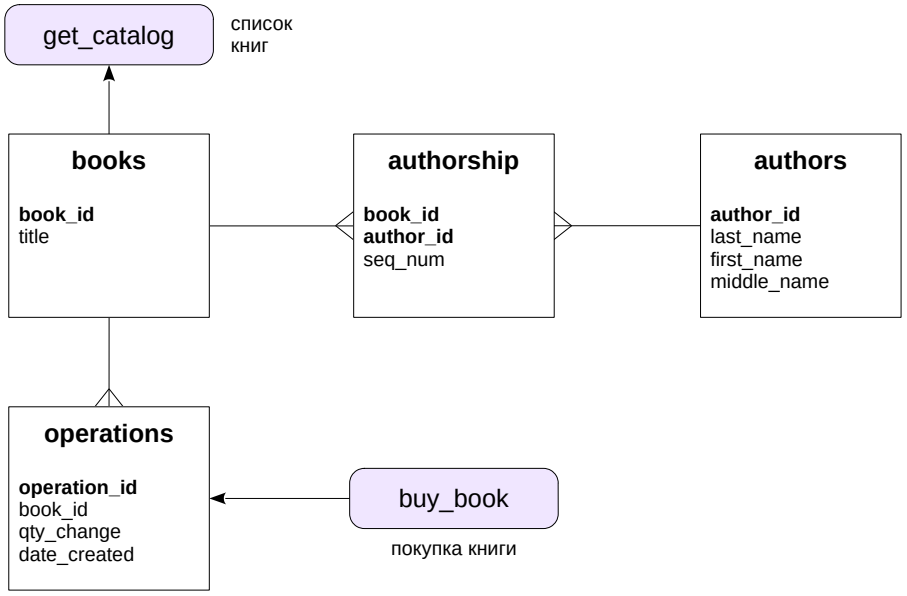
Один вариант — разрешить приложению напрямую обращаться к таблицам в базе данных и изменять их. При этом от приложения требуется детальное «знание» модели данных. Отчасти это требование можно ослабить за счет использования представлений (view).

Кроме того, от приложения требуется и определенная дисциплина — иначе очень сложно поддержать согласованность данных, защищаясь на уровне БД от любых возможных некорректных действий приложения. Но в этом случае достигается максимальная гибкость.

Другой вариант — запретить приложению доступ к таблицам и разрешить только вызовы функций. Чтение данных можно организовать с помощью табличных функций (которые возвращают набор строк). Изменение данных можно выполнять, вызывая другие функции и передавая им необходимые данные. В этом случае внутри функций можно реализовать все необходимые проверки согласованности — база данных будет защищена, но приложение сможет пользоваться только предоставленным и ограниченным набором возможностей. Такой вариант требует написания большого количества функций-оберток и может привести к потере производительности.

Вполне возможны и промежуточные варианты. Например, разрешить чтение данных непосредственно из таблиц, а изменение выполнять только через вызов функций.

Интерфейс покупателя

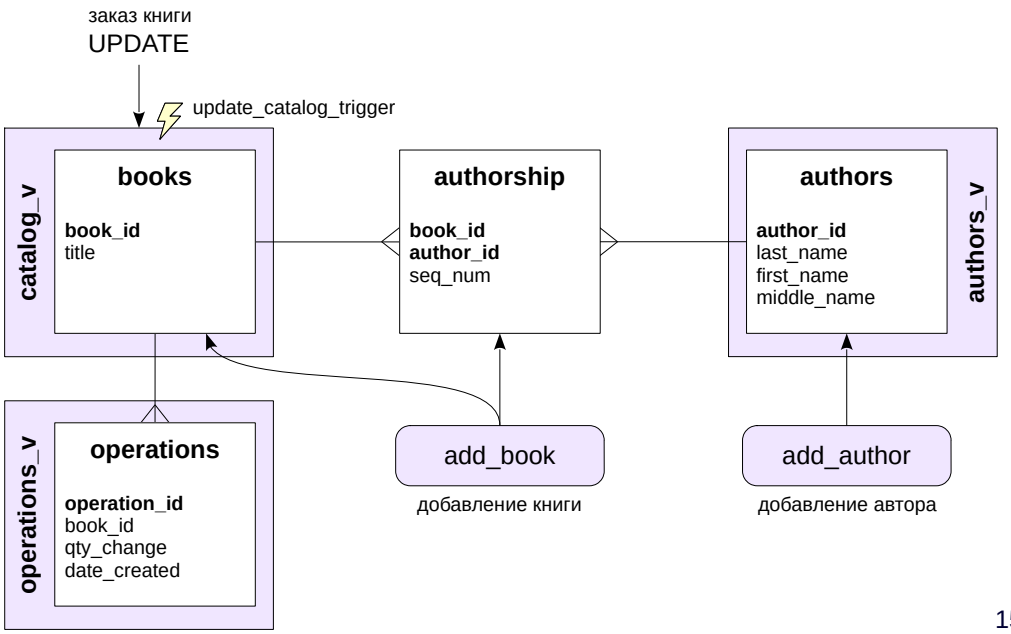


В нашем приложении мы попробуем разные варианты организации интерфейса (хотя в реальной жизни обычно лучше систематически придерживаться какого-то одного подхода).

Магазин будет использовать интерфейсные функции:

- для поиска книг — get_catalog (тема «SQL. Составные типы»);
- для покупки книг — buy_book («PL/pgSQL. Выполнение запросов»).

Интерфейс сотрудника



«Админка» для получения данных будет использовать представления (которые мы создадим в практике к этой теме):

- список книг — catalog_v;
- список авторов — authors_v;
- список операций — operations_v.

Добавление автора будет выполнять функция add_author (создадим ее в теме «PL/pgSQL. Выполнение запросов»), добавление книги — функция add_book («PL/pgSQL. Массивы»).

Для заказа книг сделаем представление catalog_v обновляемым («PL/pgSQL. Триггеры»).

Представления

Представление — запрос, у которого есть имя. Например, можно создать представление, которое показывает только авторов без отчества:

```
=> CREATE VIEW authors_no_middle_name AS
    SELECT author_id, first_name, last_name
    FROM authors
    WHERE nullif(middle_name,'') IS NULL;
```

```
CREATE VIEW
```

Теперь имя представления можно использовать в запросах практически так же, как и таблицу:

```
=> SELECT * FROM authors_no_middle_name;
```

```
 author_id | first_name | last_name
-----------+------------+-----------
         6 | Джонатан   | Свифт
(1 row)
```

В простом случае с представлением будут работать и другие операции, например:

```
=> UPDATE authors_no_middle_name SET last_name = initcap(last_name);
```

```
UPDATE 1
```

С помощью триггеров можно сделать так, чтобы и в сложных случаях для представлений работали вставка, обновление и удаление строк. Мы рассмотрим это в теме «PL/pgSQL. Триггеры».

При планировании запроса представление «разворачивается» до базовых таблиц:

```
=> EXPLAIN (costs off)
SELECT * FROM authors_no_middle_name;
```

```
                    QUERY PLAN
--------------------------------------------------
 Seq Scan on authors
   Filter: (NULLIF(middle_name, ''::text) IS NULL)
(2 rows)
```

Приложение использует три представления. Сначала они будут очень простыми, но в следующих темах мы перенесем в них часть логики приложения.

Представление для авторов — конкатенация фамилии, имени и отчества (если оно есть):

```
=> SELECT * FROM authors_v;
```

```
 author_id |         display_name
-----------+------------------------------
         1 | Пушкин Александр Сергеевич
         2 | Тургенев Иван Сергеевич
         3 | Стругацкий Борис Натанович
         4 | Стругацкий Аркадий Натанович
         5 | Толстой Лев Николаевич
         6 | Свифт Джонатан
(6 rows)
```

Представление для каталога книг — пока просто название книги:

```
=> SELECT * FROM catalog_v \gx
```

```
-[ RECORD 1 ]+-----------------------------------------------------
book_id      | 1
display_name | Сказка о царе Салтане
-[ RECORD 2 ]+-----------------------------------------------------
book_id      | 2
display_name | Муму
-[ RECORD 3 ]+-----------------------------------------------------
book_id      | 3
display_name | Трудно быть богом
-[ RECORD 4 ]+-----------------------------------------------------
book_id      | 4
display_name | Война и мир
-[ RECORD 5 ]+-----------------------------------------------------
book_id      | 5
display_name | Путешествия в некоторые удаленные страны мира в четырех частях: сочинение
Лемюэля Гулливера, сначала хирурга, а затем капитана нескольких кораблей
-[ RECORD 6 ]+-----------------------------------------------------
book_id      | 6
display_name | Хрестоматия
```

Представление для операций — дополнительно определяет тип операции (поступление или покупка):

```
=> SELECT * FROM operations_v;

 book_id |   op_type   | qty_change | date_created
---------+-------------+------------+--------------
       1 | Поступление |         10 | 24.06.2025
       1 | Поступление |         10 | 24.06.2025
       1 | Покупка     |          1 | 24.06.2025
(3 rows)
```

Итоги

Проектирование баз данных — отдельная тема

- теория важна, но не заменяет здравый смысл

Отсутствие избыточности в данных делает работу удобнее и упрощает поддержку согласованности

Для клиент-серверного интерфейса можно использовать таблицы, представления, функции, триггеры